IME 775 — Lecture 19

Convolutions in Neural Networks: 1D and 2D Convolution

---

1. Introduction: Why Convolution?

Image analysis requires identifying **local patterns** — eyes, noses, ears — rather than global properties of the entire image. Convolution is a specialized operation that examines local patterns in an input signal by sliding a small array of weights (the **kernel**) over the input.

Convolution vs. Fully Connected Layers

Property	Fully Connected (FC)	Convolutional
Connectivity	Every output connected to **all** inputs	Each output connected to a **small local neighborhood**
Weight sharing	No sharing — unique weight per connection	**Same weights** slide over the entire input
Number of parameters	$O(n_{\text{in}} \times n_{\text{out}})$ — prohibitive for images	$O(k^d)$ — depends only on kernel size
What it captures	Global relationships	**Local patterns** (edges, corners, textures)

Parameter definitions for the table above:

Symbol	Definition
n_{in}	Number of input units (e.g., total pixels in an image)
n_{out}	Number of output units
k	Kernel size — the number of weights along one spatial dimension

d	Number of spatial dimensions (1 for 1D signals, 2 for images)

Key insight: In a multilayer convolutional neural network, the lowest layers detect simple local features (edges, corners), and successive layers combine them into increasingly complex, increasingly global patterns (ears → faces → people).

---

2. One-Dimensional Convolution

2.1 Graphical View

Imagine a stretched rope (the **input array**) over which a measuring ruler (the **kernel**) slides. At each position (slide stop), we:

1. Multiply each input element by the kernel element resting on it
2. Sum the products → one output element

As the ruler slides left-to-right, a 1D output array is generated.

2.2 Convolution Parameters

Parameter	Symbol	Description
Input size	n	Length of the 1D input array
Kernel size	k	Length of the weight array
Stride	s	Number of input elements the kernel shifts per step
Padding	p	How to handle kernel elements falling outside the input
Output size	o	Length of the resulting output array

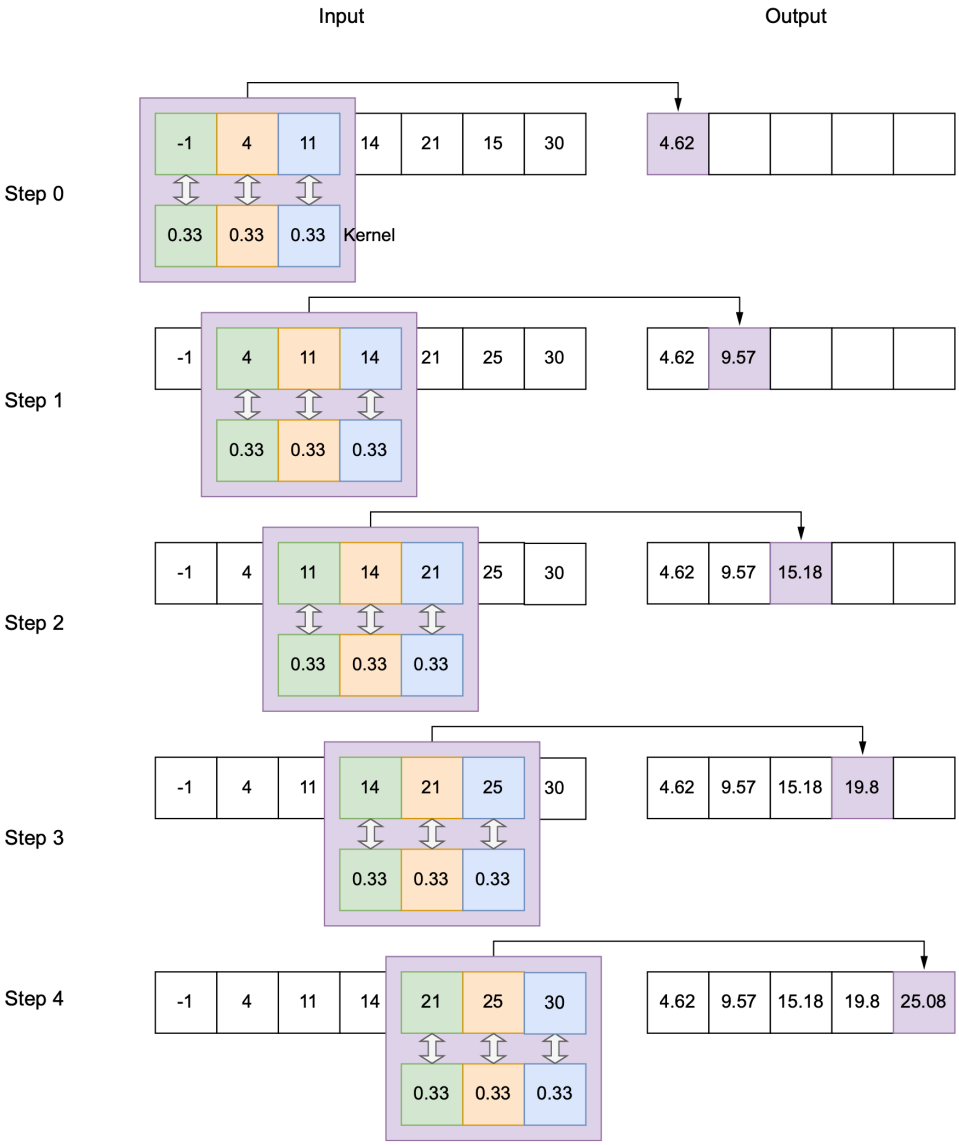


Figure 10.1 1D convolution with a local averaging kernel of size 3, stride 1, and valid padding on the input array of size 7

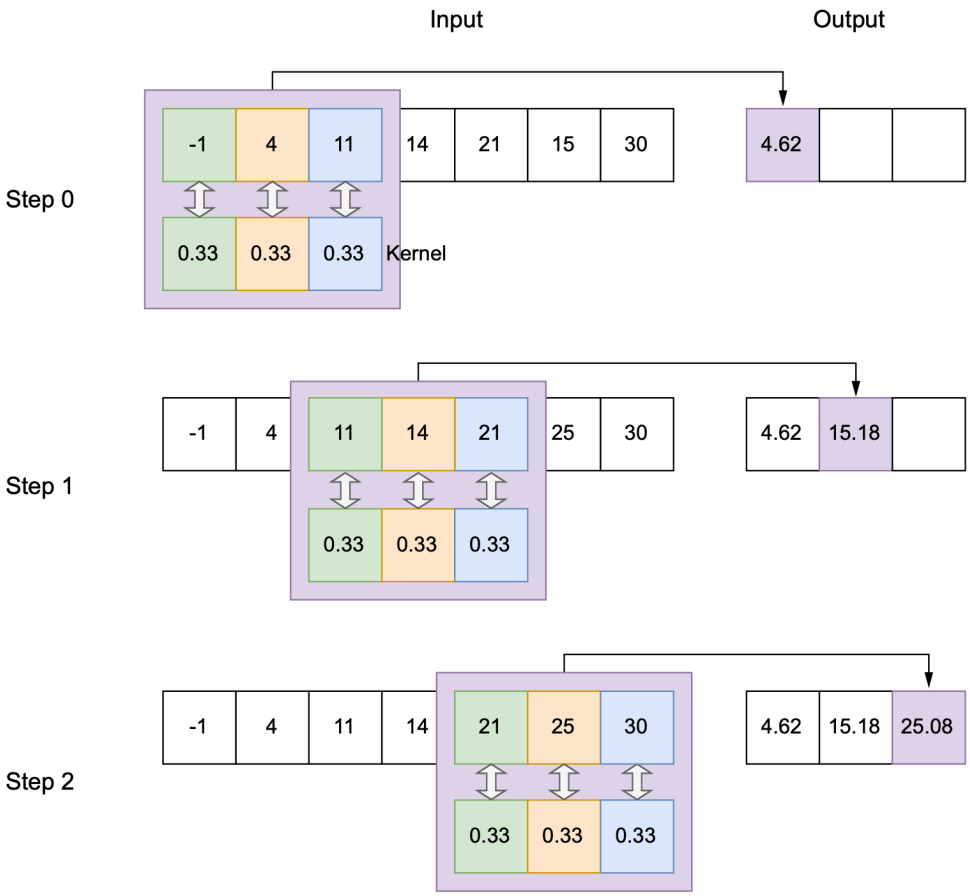


Figure 10.2 1D convolution with a local averaging kernel of size 3, stride 2, and valid padding

2.3 Padding Strategies

Valid padding: Stop whenever any kernel element falls outside the input. The entire kernel always falls on valid input elements. Output is smaller than input.

Same (zero) padding: Continue sliding until the kernel's left end reaches the last input element. Ghost input elements outside the array boundary are set to 0. With stride 1, output size = input size.

2.4 The Convolution Equation

$$Y_x = \sum_{j=0}^{k_W-1} X_{x+j}\, W_j \qquad \forall\, x \in S_o$$

Reading the equation piece by piece:

Symbol	Meaning
Y_x	The output value at position x
$\sum_{j=0}^{k_W-1}$	Sum over all kernel positions $j = 0, 1, \ldots, k_W - 1$
X_{x+j}	The input element at index $x + j$ — this is the input pixel that kernel element j is currently covering
W_j	The j-th weight of the kernel
$X_{x+j}\,, W_j$	Multiply each covered input element by its corresponding kernel weight
$\forall\, x \in S_o$	Repeat for every valid output position x in the output grid S_o

In plain language: Place the kernel's left edge at input position x. The kernel covers input elements $X_x, X_{x+1}, \ldots, X_{x+k_W-1}$. Multiply each covered input element by the kernel weight sitting on it, and sum the products. That sum is the single output value Y_x. Then slide the kernel to the next position (determined by stride) and repeat.

Example: Input $\vec{x} = [4, 1, 7, 3, 5]$, kernel $\vec{w} = [W_0, W_1, W_2] = [\frac{1}{3}, \frac{1}{3}, \frac{1}{3}]$, stride 1, valid padding.

With $k_W = 3$, the output positions are $S_o = \{0, 1, 2\}$. Expanding the sum for each:

$$Y_0 = \underbrace{X_0 W_0}_{4\cdot\frac{1}{3}} + \underbrace{X_1 W_1}_{1\cdot\frac{1}{3}} + \underbrace{X_2 W_2}_{7\cdot\frac{1}{3}} = \frac{12}{3} = 4.0$$

$$Y_1 = \underbrace{X_1 W_0}_{1\cdot\frac{1}{3}} + \underbrace{X_2 W_1}_{7\cdot\frac{1}{3}} + \underbrace{X_3 W_2}_{3\cdot\frac{1}{3}} = \frac{11}{3} \approx 3.67$$

$$Y_2 = \underbrace{X_2 W_0}_{7\cdot\frac{1}{3}} + \underbrace{X_3 W_1}_{3\cdot\frac{1}{3}} + \underbrace{X_4 W_2}_{5\cdot\frac{1}{3}} = \frac{15}{3} = 5.0$$

Output: $\vec{y} = [4.0,\ 3.67,\ 5.0]$. Notice how the index into $\vec{x}$ shifts by 1 (the stride) between successive outputs, but the kernel weights W_0, W_1, W_2 stay the same — this is the "sliding" and "weight sharing" at work.

---

3. Output Size Formula

For an input of size n, kernel size k, stride s, and zero-padding p on each side:

$$o = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

Workout: An input of size $n = 32$, kernel $k = 5$, stride $s = 2$, valid padding ($p = 0$). What is the output size?

Solution:

$$o = \left\lfloor \frac{32 - 5}{2} \right\rfloor + 1 = \lfloor 13.5 \rfloor + 1 = 13 + 1 = 14$$

---

4. 1D Convolution Applications

4.1 Curve Smoothing (Local Averaging)

A kernel with **uniform normalized weights** $\vec{w} = \left[\frac{1}{3},\ \frac{1}{3},\ \frac{1}{3}\right]$ computes the moving average of successive sets of three input values.

Effect: The output is a **smoothed** version of the input — it captures the low-frequency, long-term broad trend while eliminating high-frequency, short-term noise.

4.2 Edge Detection

A kernel with **antisymmetric weights** $\vec{w} = \left[\frac{1}{2}, -\frac{1}{2}\right]$ detects sharp changes in the input.

Effect: The output **spikes** at locations where the input values change abruptly (edges) and is near zero in flat regions. Edges provide vital semantic clues for understanding signals.

```
import torch
import torch.nn as nn

# --- 1D Smoothing Convolution ---
x = torch.tensor([14.0, -1.0, 4.0, 11.0, 21.0, 25.0, 30.0])
w = torch.tensor([1/3, 1/3, 1/3])

x = x.unsqueeze(0).unsqueeze(0)  # Shape: [1, 1, 7] (N x C x L)
w = w.unsqueeze(0).unsqueeze(0)  # Shape: [1, 1, 3]

conv1d = nn.Conv1d(1, 1, kernel_size=3, stride=1, padding=0,
bias=False)
conv1d.weight = nn.Parameter(w, requires_grad=False)
with torch.no_grad():
    y_smooth = conv1d(x)
print("Smoothed output:", y_smooth)

# --- 1D Edge Detection ---
x_edge = torch.tensor([10.0, 10.0, 10.0, 10.0, 51.0, 51.0, 51.0,
51.0, 49.0, 9.0, 9.0])
w_edge = torch.tensor([0.5, -0.5])

x_edge = x_edge.unsqueeze(0).unsqueeze(0)
w_edge = w_edge.unsqueeze(0).unsqueeze(0)

conv_edge = nn.Conv1d(1, 1, kernel_size=2, stride=1, padding=0,
bias=False)
conv_edge.weight = nn.Parameter(w_edge, requires_grad=False)
with torch.no_grad():
    y_edge = conv_edge(x_edge)
print("Edge detection output:", y_edge)
```

5. 1D Convolution as Matrix Multiplication

The convolution can be expressed as a multiplication of a **block-diagonal weight matrix** W with the input vector $\vec{x}$:

$$\vec{y} = \vec{w} \circledast \vec{x} = W\vec{x}$$

For kernel size 3, stride 1, valid padding:

$$W = \begin{bmatrix} w_0 & w_1 & w_2 & 0 & 0 & \cdots & 0 \\ 0 & w_0 & w_1 & w_2 & 0 & \cdots & 0 \\ \vdots & & & & & \ddots & \vdots \\ 0 & 0 & \cdots & 0 & w_0 & w_1 & w_2 \end{bmatrix}$$

Key observations:

- The matrix is **sparse** and **block-diagonal** — each row has only k nonzero entries
- The kernel weights shift rightward by s positions in successive rows (simulating sliding)
- For stride 2, the shift between successive rows is 2 positions instead of 1
- Forward propagation ($\vec{y} = W\vec{x}$) and backpropagation ($\nabla_{\vec{x}} = W^T \nabla_{\vec{y}}$) work exactly as with FC layers

---

6. Two-Dimensional Convolution

6.1 Why 2D Convolution?

Images are 2D arrays of pixels. Rasterizing a 2D image into a 1D vector **destroys spatial neighborhoods** — pixels that are vertically adjacent in the image become far apart in the rasterized array. Therefore, 2D convolution must be a specialized operation that preserves 2D neighborhoods.

6.2 Graphical View

Imagine a wall (the **input image**) over which a tile (the **2D kernel**) slides:

1. The tile slides left-to-right across the first row of positions
2. Then drops down by the vertical stride and repeats
3. At each slide stop, multiply element-wise and sum → one output pixel
4. The result is a 2D output array (feature map)

6.3 2D Convolution Parameters

Parameter	Symbol	Description
Input size	$[H, W]$	Height and width of input image
Kernel size	$[k_H, k_W]$	Height and width of the kernel
Stride	$[s_H, s_W]$	Vertical and horizontal stride
Output size	$[o_H, o_W]$	Computed per dimension using the formula from Section 3

6.4 The 2D Convolution Equation

$$Y_{y,x} = \sum_{i=0}^{k_H-1} \sum_{j=0}^{k_W-1} X_{y+i,\,x+j}\, W_{i,j} \qquad \forall\,(y,x) \in S_o$$

Workout: A 5×5 image is convolved with a 3×3 kernel, stride $[1, 1]$, valid padding. What is the output size?

Solution: Apply the formula per dimension:

$$o_H = \left\lfloor \frac{5-3}{1} \right\rfloor + 1 = 3, \qquad o_W = \left\lfloor \frac{5-3}{1} \right\rfloor + 1 = 3$$

Output size: 3×3.

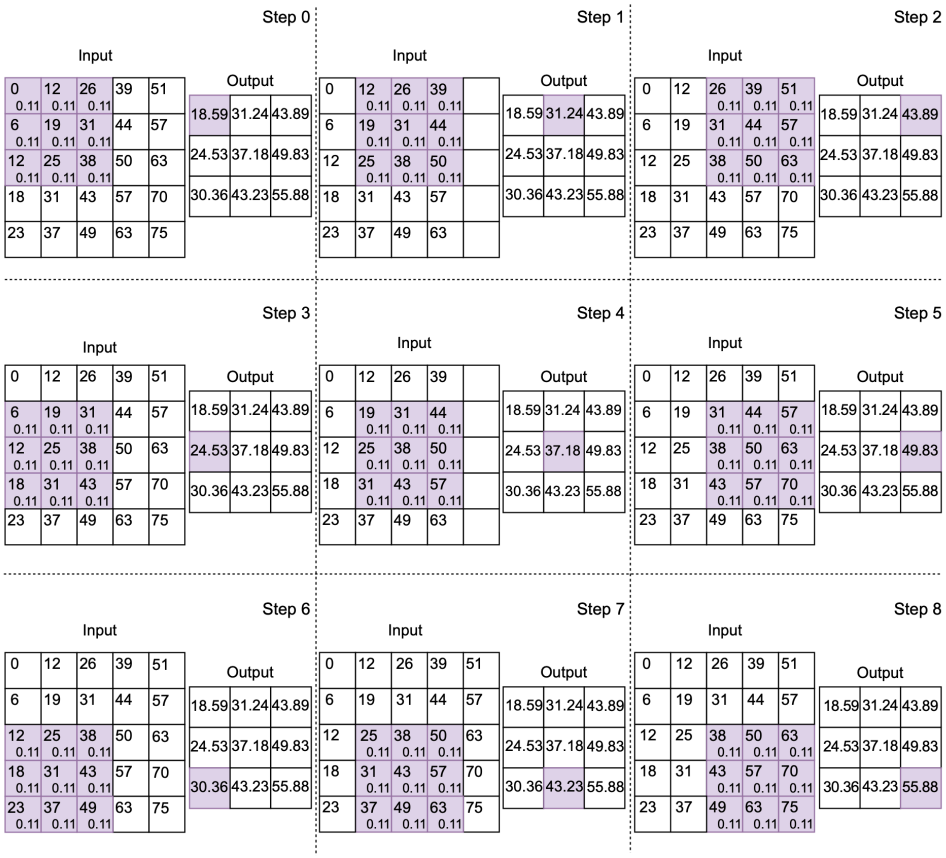


Figure 10.7 **2D convolution with a local averaging kernel of size $[3, 3]$, stride $[1, 1]$, and valid padding. Each pixel is shown as a small rectangle, with the pixel's gray level written in the rectangle. The shaded rectangle identifies the current location of the kernel. The kernel is sliding over the input in raster order. Successive steps indicate slide stops. For each pixel that is overlapped by the kernel, the weight of the kernel element falling on it is written in small font.**

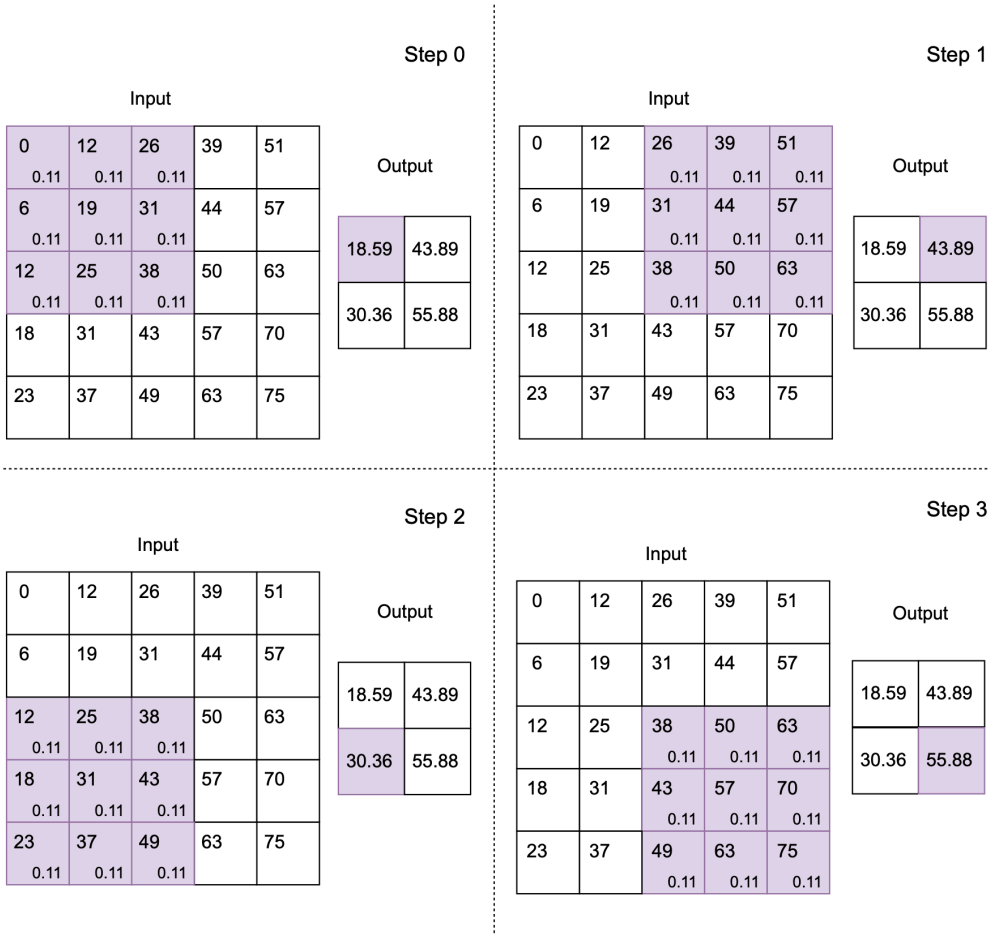


Figure 10.8 2D convolution with a local averaging kernel of size $[3, 3]$, stride $[2, 2]$, and valid padding

---

7. 2D Convolution Applications

7.1 Image Smoothing (Denoising)

A 3×3 uniform kernel:

$$W = \begin{bmatrix} \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \\ \frac{1}{9} & \frac{1}{9} & \frac{1}{9} \end{bmatrix}$$

Each output pixel is the **local average** of a 3×3 neighborhood. This eliminates salt-and-pepper noise while preserving broad image content.

(a) Input image (b) Smoothed/denoised output image

7.2 Edge Detection

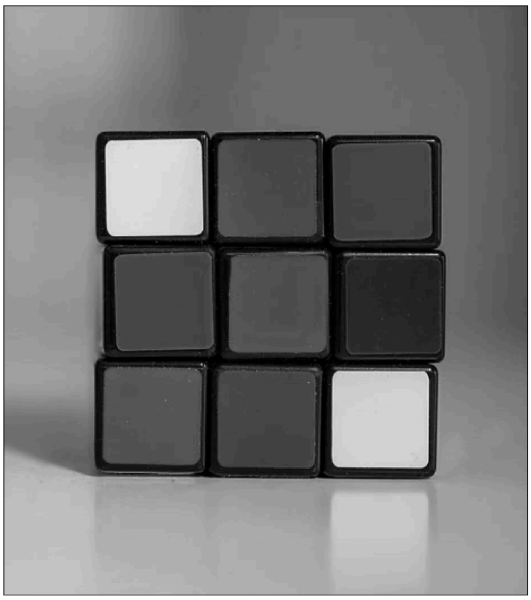
Vertical edge detection kernel:

$$W_{\text{vert}} = \begin{bmatrix} -0.25 & 0.25 \\ -0.25 & 0.25 \end{bmatrix}$$

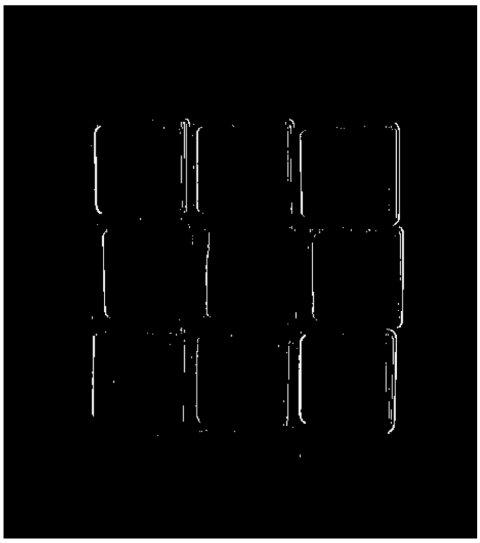
Horizontal edge detection kernel:

$$W_{\text{horiz}} = \begin{bmatrix} -0.25 & -0.25 \\ 0.25 & 0.25 \end{bmatrix}$$

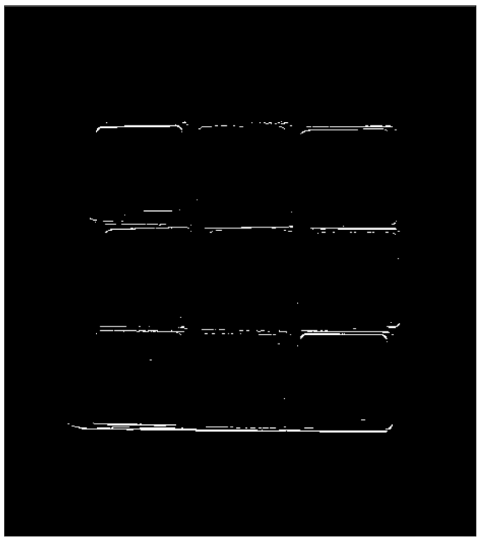
How it works: In a uniform region, the positive and negative kernel elements fall on equal values → weighted sum is zero (suppressed). At an edge, one half falls on high values and the other on low values → large output (detected).



(a) Input image



(b) Vertical edges detected by applying 2D



(c) Horizontal edges detected by applying 2D

```
import torch
import torch.nn as nn

# --- 2D Smoothing (Local Averaging) ---
x = torch.tensor([
    [0.0, 6.0, 12.0, 18.0, 23.0],
    [12.0, 19.0, 25.0, 31.0, 37.0],
    [26.0, 31.0, 38.0, 43.0, 49.0],
```

```
    [39.0, 44.0, 50.0, 57.0, 63.0],
    [51.0, 57.0, 63.0, 70.0, 75.0],
])
w_smooth = torch.full((3, 3), 1/9)

x_4d = x.unsqueeze(0).unsqueeze(0)         # [1, 1, 5, 5]
w_4d = w_smooth.unsqueeze(0).unsqueeze(0)   # [1, 1, 3, 3]

conv2d = nn.Conv2d(1, 1, kernel_size=3, stride=1, bias=False)
conv2d.weight = nn.Parameter(w_4d, requires_grad=False)
with torch.no_grad():
    y_smooth = conv2d(x_4d)
print("Smoothed output:\n", y_smooth.squeeze())

# --- 2D Vertical Edge Detection ---
x_edge = torch.tensor([
    [100.0, 100.0, 100.0, 100.0],
    [100.0, 100.0, 100.0, 100.0],
    [10.0,  10.0,  100.0, 100.0],
    [10.0,  10.0,  100.0, 100.0],
])
w_vedge = torch.tensor([[-0.25, 0.25], [-0.25, 0.25]])

x_4d = x_edge.unsqueeze(0).unsqueeze(0)
w_4d = w_vedge.unsqueeze(0).unsqueeze(0)

conv_vedge = nn.Conv2d(1, 1, kernel_size=2, stride=1, bias=False)
conv_vedge.weight = nn.Parameter(w_4d, requires_grad=False)
with torch.no_grad():
    y_vedge = conv_vedge(x_4d)
print("Vertical edge output:\n", y_vedge.squeeze())
```

---

8. PyTorch Tensor Conventions

PyTorch convolution layers expect specific tensor shapes:

Dimension	1D Conv (`Conv1d`)	2D Conv (`Conv2d`)

Input	$N \times C \times L$	$N \times C \times H \times W$
Kernel	$C_{\text{out}} \times C_{\text{in}} \times k_W$	$C_{\text{out}} \times C_{\text{in}} \times k_H \times k_W$

where:

- N = batch size (number of input instances)
- C = number of channels (1 for grayscale, 3 for RGB)
- L = sequence length; H, W = height, width

Use `torch.unsqueeze()` to add the batch and channel dimensions to raw tensors.

---

9. 2D Convolution as Matrix Multiplication

For a 4×4 input image and a 2×2 kernel with stride $[1, 1]$ and valid padding:

1. **Rasterize** the 4×4 image into a 16×1 vector
2. Construct a 9×16 block-diagonal weight matrix
3. Each row places the 4 kernel weights at positions corresponding to one slide stop (with zeros elsewhere)
4. The 9-element output vector folds back into a 3×3 output image

$$W_{\text{conv}} = \begin{bmatrix} w_{0,0} & w_{0,1} & 0 & 0 & w_{1,0} & w_{1,1} & 0 & 0 & 0 & \cdots & 0 & & & \\ 0 & w_{0,0} & w_{0,1} & 0 & 0 & w_{1,0} & w_{1,1} & 0 & 0 & \cdots & 0 & & & \\ \vdots & & & & & & & & & \ddots & \vdots & & & \\ 0 & \cdots & 0 & 0 & 0 & 0 & 0 & 0 & w_{0,0} & w_{0,1} & 0 & 0 & w_{1,0} & w_{1,1} \end{bmatrix}$$

The same block-diagonal structure as 1D, but with gaps within each row corresponding to the image width. Forward and backpropagation proceed exactly as with FC layers.

---

10. Functional API

PyTorch also provides `torch.nn.functional.conv1d` and `torch.nn.functional.conv2d` for direct invocation without creating a layer object:

```
import torch
import torch.nn.functional as F

x = torch.tensor([10.0, 10.0, 10.0, 10.0, 51.0, 51.0, 51.0, 51.0,
49.0, 9.0, 9.0])
w = torch.tensor([0.5, -0.5])

x = x.unsqueeze(0).unsqueeze(0)
w = w.unsqueeze(0).unsqueeze(0)

y = F.conv1d(x, w, stride=1)
print("Functional conv1d output:", y)
```

Key Takeaways

1. **Convolution** extracts local patterns by sliding a kernel over the input — unlike FC layers, which connect every input to every output
2. **Shared weights** drastically reduce parameters and ensure the same pattern is detected everywhere in the input
3. **1D convolution** slides a ruler over a rope; applications include **smoothing** (uniform kernel) and **edge detection** (antisymmetric kernel)
4. **Output size** is governed by: $o = \lfloor (n + 2p - k)/s \rfloor + 1$
5. **2D convolution** slides a tile over a wall; it is essential for images because rasterization destroys 2D neighborhoods
6. **Smoothing kernels** (uniform weights) eliminate noise; **edge-detection kernels** (antisymmetric weights) highlight sharp transitions
7. Convolution can be viewed as **sparse, block-diagonal matrix multiplication** — backpropagation works identically to FC layers

8. PyTorch expects tensors in $N \times C \times \text{spatial dims}$ format